

Vorlesung

Theoretische Informatik

(02 Endliche Automaten, KMP-Algorithmus)

Alois Heinz
Hochschule Heilbronn,
Max-Planck-Str. 39, 74081 Heilbronn
heinz@hs-heilbronn.de

17/20. März 2026

Motivation

- Einfacher Automat (Fahrkarten)
- Definierter Ablauf von Aktionen
- Determinierte Folge von Aktionen

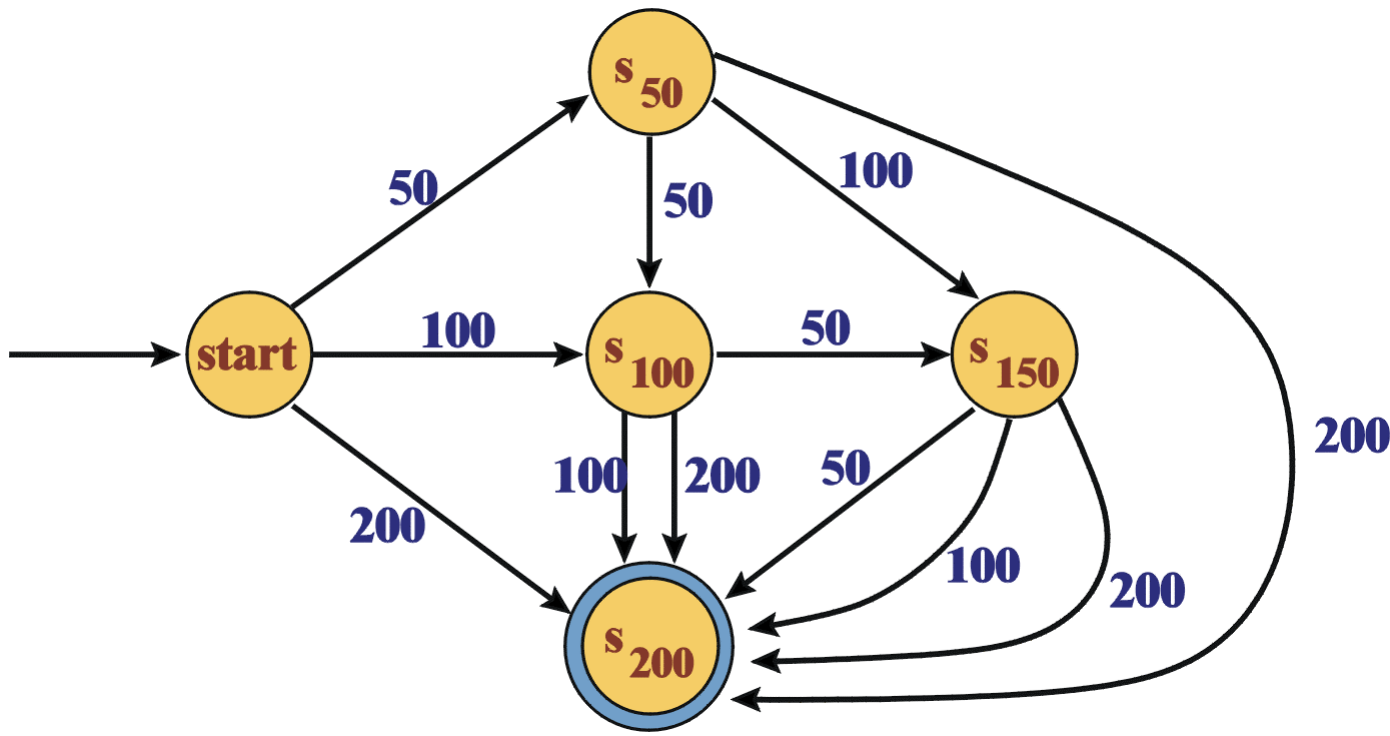
Ausstattung eines Automaten A

1. Eingabe: A wird von außen mit **Eingabedaten** versorgt
2. Interne Zustände: A befindet sich immer in einem definierten **Zustand**. Er kann **Zustandsübergänge** durchführen, etwa unter Einfluss der Eingabe.
3. Ausgabe: A kann unter gewissen Umständen **Ausgabeinformationen** erzeugen.

Beispiel: Schwimmbadautomat

Eigenschaften von A_{swim} :

- Münzeingaben mit Werten 50, 100, 200 können in beliebiger Reihenfolge erfolgen
- Nach Einwurf von insgesamt ≥ 200 akzeptiert A_{swim} : der Eintritt kann erfolgen
- Der Gesamtwert der bisherigen Eingaben wird vermerkt im aktuellen Zustand

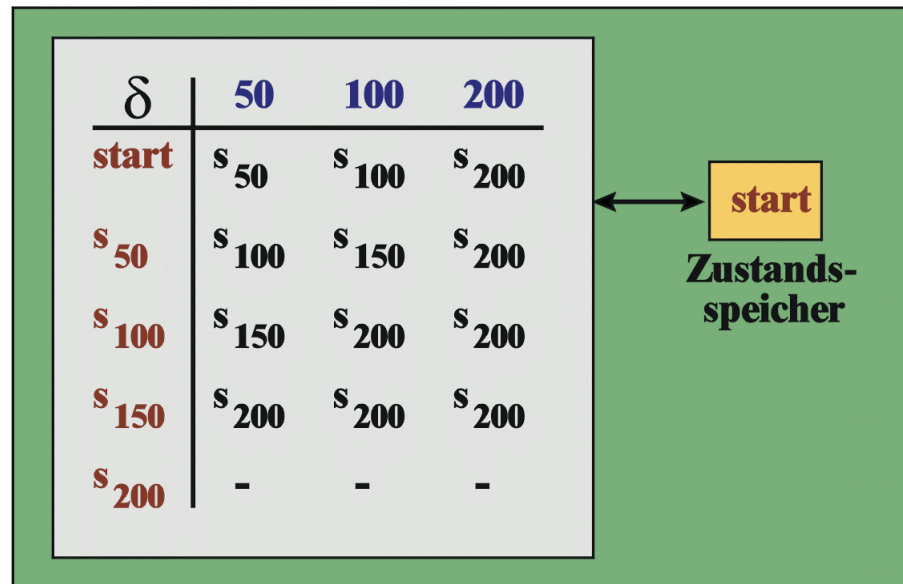
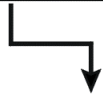


Schwimmbadautomat in Start-Konfiguration

- **Speicher:** Nur lesbares **Eingabeband** enthält Eingaben als Folge von Zeichen, **Zustandsspeicher** enthält aktuellen Zustand
- **Programm, Kontrolle:** Zustandsüberföhrungsfunktion δ

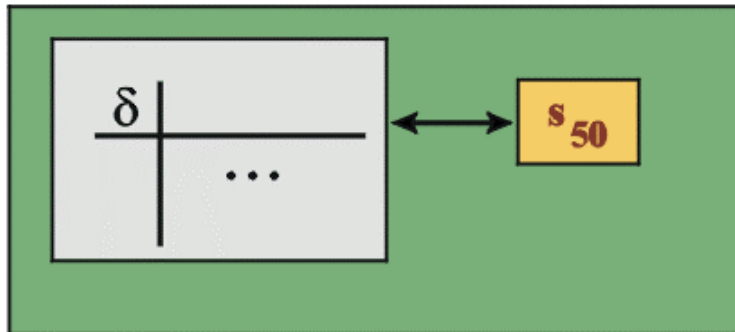
Eingabeband

50	100	50		...
----	-----	----	--	-----

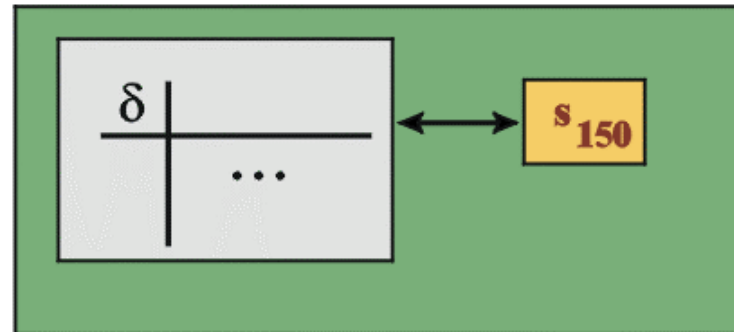


Berechnung des Schwimmbadautomaten

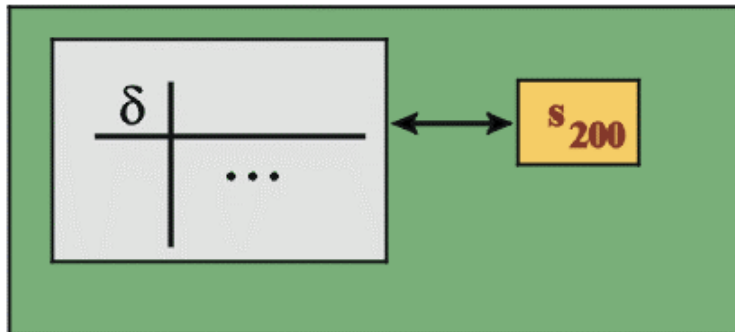
50	100	50		...
----	-----	----	--	-----



50	100	50		...
----	-----	----	--	-----



50	100	50		...
----	-----	----	--	-----



Alphabete

- Automaten verarbeiten **Zeichenfolgen**, die aus atomaren **Symbolen** bestehen.
- Menge aller zugelassenen Zeichen: **Endliches Alphabet Σ**

Beispiele:

- 1) $\Sigma = \{\underline{50}, \underline{100}, \underline{200}\} \quad ; |\Sigma| = 3$
- 2) $\Sigma = \{a_1, a_2, \dots, a_n\} \quad ; |\Sigma| = n$
- 3) $\Sigma = \emptyset \quad ; |\Sigma| = 0$
- 4) $\Sigma = \{a, b, \dots, z\} \quad ; |\Sigma| = 26$
- 5) $\Sigma = \{0, 1\} \quad ; |\Sigma| = 2$

Wörter

- Wort über Σ : Zeichenfolge mit Elementen aus Σ
- Erzeugung eines Wortes über Σ : Hintereinanderreihung (Konkatenation) von Zeichen (oder Wörtern) aus Σ
- Menge aller Wörter über Σ : Σ^*

Definition von Σ^* :

1. Jedes Zeichen $a \in \Sigma$ ist ein Wort: $a \in \Sigma \Rightarrow a \in \Sigma^*$
2. Die Konkatenation zweier Wörter ist ein Wort: $v, w \in \Sigma^* \Rightarrow vw \in \Sigma^*$
3. Das leere Wort ϵ ist in Σ^* : $\epsilon \in \Sigma^*$
Es gilt: $w \in \Sigma^* \Rightarrow \epsilon w = w\epsilon = w$

Definition von Σ^+ : $\Sigma^+ := \Sigma^* \setminus \epsilon$

Lexikographische Ordnung

Für $\Sigma = \{a, b\}$ ist

$$\begin{aligned}\Sigma^* &= \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bab, bba, bbb, \dots\} \\ &= \{\epsilon, a, b, a^2, ab, ba, b^2, a^3, a^2b, aba, ab^2, ba^2, bab, b^2a, b^3, \dots\}\end{aligned}$$

Schreibweise: $a^n = \underbrace{aa \dots a}_{n\text{-mal}}$ n -te Potenz von a

Für $w = xyz$ mit $x, y, z \in \Sigma^*$ heißt x **Präfix**, y **Infix**, z **Postfix** oder **Suffix** von w .

Die Relation **Lexikographische Ordnung** $\prec \subseteq \Sigma^* \times \Sigma^*$ ist wie folgt definiert:

Sei $v = v_1 \dots v_m$, $w = w_1 \dots w_n$ mit $v_i, w_j \in \Sigma$ für $1 \leq i \leq m$ und $1 \leq j \leq n$.

Dann ist $v \prec w :\Leftrightarrow$

1. $m < n$, d.h. $|v| < |w|$, oder
2. $m = n$ und $v_1 \dots v_k = w_1 \dots w_k$ und $v_{k+1} < w_{k+1}$ für ein $k \in \{1, \dots, m\}$

Für $\Sigma = \{a, b\}$ ist z.B. $bb \prec aaa$ und $baab \prec babb$ (wegen $a < b$)

Wortfunktionen

Wortfunktionen sind Funktionen, die Wörter als Argumente haben.

$$\begin{aligned} \text{substr} &: \Sigma^* \times \Sigma^* \rightarrow \mathbb{B} \quad \text{def. durch:} \\ \text{substr}(w, v) &= \begin{cases} 1 & \text{falls } v \text{ Infix von } w \\ 0 & \text{sonst} \end{cases} \end{aligned}$$

Die Teilworterkennung **substr** kann durch endliche Automaten in linearer Zeit erfolgen.

$$\begin{aligned} \ell &: \Sigma^* \rightarrow \mathbb{N}_0 \quad \text{def. durch:} \\ \ell(\epsilon) &= 0 \\ \ell(wa) &= \ell(w) + 1 \quad \text{für } w \in \Sigma^*, a \in \Sigma \end{aligned}$$

$$\begin{aligned} \text{anzahl} &: \Sigma^* \times \Sigma \rightarrow \mathbb{N}_0 \quad \text{def. durch:} \\ \text{anzahl}(\epsilon, b) &= 0 \quad \text{für alle } b \in \Sigma \\ \text{anzahl}(wa, b) &= \begin{cases} \text{anzahl}(w, b) + 1 & \text{falls } a = b, \text{ für } a, b \in \Sigma, w \in \Sigma^* \\ \text{anzahl}(w, b) & \text{falls } a \neq b, \text{ für } a, b \in \Sigma, w \in \Sigma^* \end{cases} \end{aligned}$$

Formale Sprachen und Operationen

Eine **formale Sprache** L über Σ ist jede Wortmenge mit $L \subseteq \Sigma^*$.

Operationen auf Wortmengen sind Vereinigung $\cup$, Durchschnitt $\cap$, Differenz $\setminus$

Die **Konkatenation** $\circ$ ist eine weitere Operation mit:

$$L_1 \circ L_2 = \{vw \mid v \in L_1, w \in L_2\}$$

Alle Wörter aus L_1 werden mit sämtlichen Wörtern aus L_2 verknüpft.

Die n -te **Potenz** einer Sprache L ist definiert durch:

$$\begin{aligned} L^0 &= \{\epsilon\} \\ L^{n+1} &= L^n \circ L \end{aligned}$$

Das **Kleene-Stern-Produkt** L^* ist die Vereinigung aller Potenzen von L :

$$L^* = \bigcup_{n \geq 0} L^n = L^0 \cup L^1 \cup L^2 \cup \dots \quad \text{Und } L^+ = L^* \setminus L^0$$

Deterministische endliche Automaten

Ein **Deterministischer Endlicher Automat** (DFA) ist ein Quintupel

$$A = (\Sigma, S, \delta, s_0, F)$$

Dabei ist Σ das endliche **Eingabealphabet**, S die **Zustandsmenge**, $s_0 \in S$ der **Startzustand**, $F \subseteq S$ die Menge der **Endzustände** und $\delta : S \times \Sigma \rightarrow S$ die **Zustandsüberföhrungsfunktion**.

Bem:

- δ ist eine Funktion, daher das Attribut deterministisch
- δ muss nicht total (überall definiert) sein
- Schreibt man δ als Menge von Tripeln (s, a, t) mit $\delta(s, a) = t$, so kann es als Programm des DFA verstanden werden

Verhalten von deterministischen endlichen Automaten

Erweitere Zustandsüberföhrungsfunktion von Zeichen auf W6rter:

$$\begin{aligned}\delta^* &: S \times \Sigma^* \rightarrow S \quad \text{def. durch:} \\ \delta^*(s, \epsilon) &= s \quad \text{f\u00fcr alle } s \in S \\ \delta^*(s, aw) &= \delta^*(\delta(s, a), w) \quad \text{f\u00fcr alle } a \in \Sigma, w \in \Sigma^*\end{aligned}$$

δ^* bestimmt den Zustand nach Abarbeitung eines Wortes aus einem Anfangszustand.

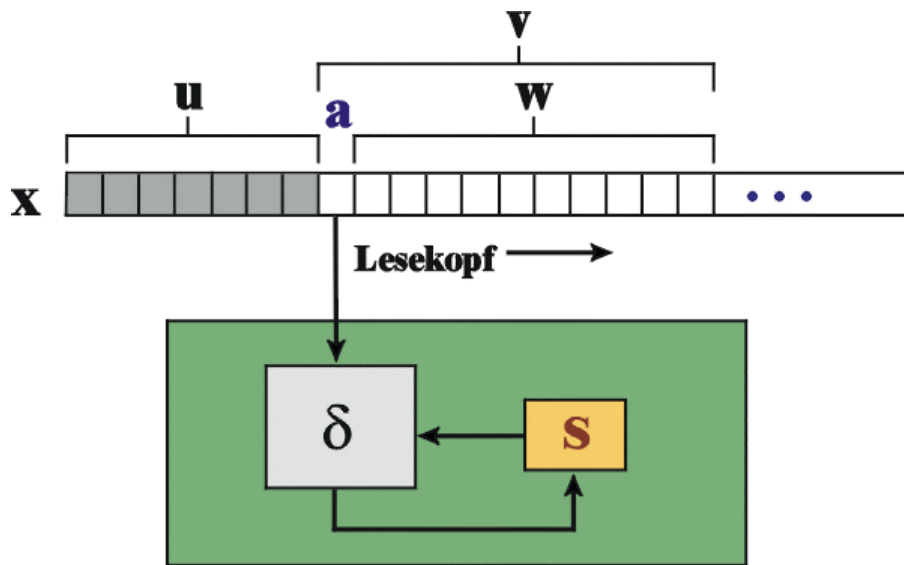
Bsp:

$$\begin{aligned}\delta^*(s_{50}, \underline{50} \underline{50} \underline{100}) &= \delta^*(\delta(s_{50}, \underline{50}), \underline{50} \underline{100}) \\ &= \delta^*(s_{100}, \underline{50} \underline{100}) \\ &= \delta^*(\delta(s_{100}, \underline{50}), \underline{100}) \\ &= \delta^*(s_{150}, \underline{100}) \\ &= \delta^*(\delta(s_{150}, \underline{100}), \epsilon) \\ &= \delta^*(s_{200}, \epsilon) \\ &= s_{200}\end{aligned}$$

Konfigurationen

Eine **Konfiguration** k_A eines DFA A

- beschreibt den aktuellen Verarbeitungsstand
- ist definiert als Paar (s, v) mit $s \in S$ (aktueller Zustand),
 $v \in \Sigma^*$ (Restinschrift des Eingabebandes); Lesekopf steht auf erstem Zeichen von v



Die Menge aller möglichen Konfigurationen ist $K = S \times \Sigma^*$

Konfigurationsübergänge

Ein Konfigurationsübergang

$$(s, aw) \vdash (t, w)$$

kann stattfinden, wenn $\delta(s, a) = t$

Die **Abarbeitung** eines Wortes $x = x_1x_2 \dots x_r$ durch einen DFA kann als Folge von Zustandsübergängen beschrieben werden:

$$(s_0, x_1x_2 \dots x_r) \vdash (s_1, x_2 \dots x_r) \vdash \dots \vdash (s_r, \epsilon)$$

Mit $\vdash^*$ wird wieder die transitiv-reflexive Hülle von $\vdash$ beschrieben.

Bsp.:

$$\begin{array}{ccc} (start, \underline{50} \ \underline{100} \ \underline{50}) & \vdash & (s_{50}, \underline{100} \ \underline{50}) \vdash (s_{150}, \underline{50}) \vdash (s_{200}, \epsilon) \\ (start, \underline{50} \ \underline{100} \ \underline{50}) & \vdash^* & (s_{200}, \epsilon) \end{array}$$

Reguläre Sprachen

- Für einen DFA $A = (\Sigma, S, \delta, s_0, F)$ ist

$$L(A) = \{w \in \Sigma^* \mid (s_0, w) \vdash^* (s, \epsilon), s \in F\}$$

die von A **akzeptierte Sprache**

- Eine Sprache $L \subseteq \Sigma^*$ heißt **regulär**, falls es einen DFA A gibt mit $L(A) = L$
- Für ein Alphabet Σ bezeichnet DFA_Σ die Menge der von DFA akzeptierten Sprachen über Σ und REG_Σ die Menge der regulären Sprachen über Σ .
(Es gilt: $DFA_\Sigma = REG_\Sigma$)
- Zwei DFA A und A' heißen **äquivalent**, falls sie die gleiche Sprache akzeptieren, also wenn $L(A) = L(A')$

Bem: $L(A)$ könnte äquivalent wie folgt definiert werden:

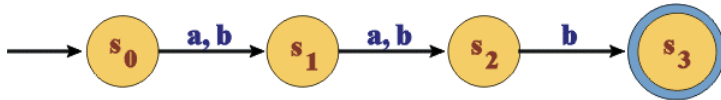
$$L(A) = \{w \in \Sigma^* \mid \delta^*(s_0, w) \in F\}$$

Beispiel: DFA für L_{3b}

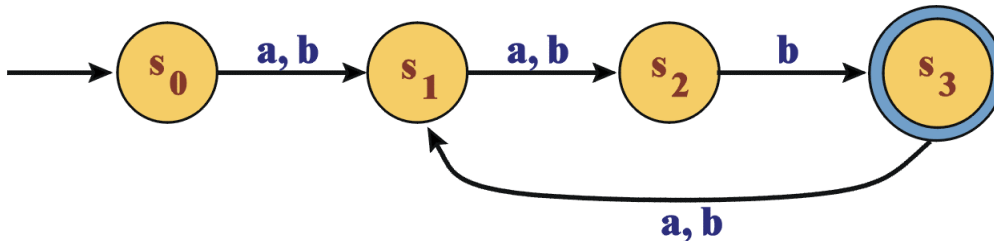
Synthese-Aufgabe: Finde DFA für

$$L_{3b} = \{w \in \{a, b\}^* \mid w = w_1 \dots w_n, w_i \in \{a, b\}, w_{3i} \neq a, 1 \leq i \leq n, n \geq 0\}$$

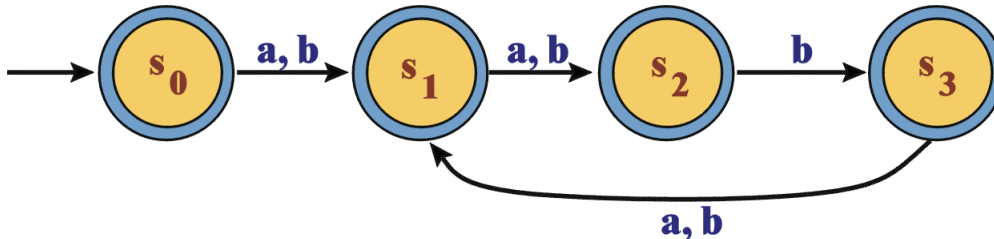
- Dreier-Gruppe von Zeichen, deren letztes b ist, muss erkannt werden:



- Beliebig viele Dreier-Gruppen derselben Art sind als Präfixe erlaubt:



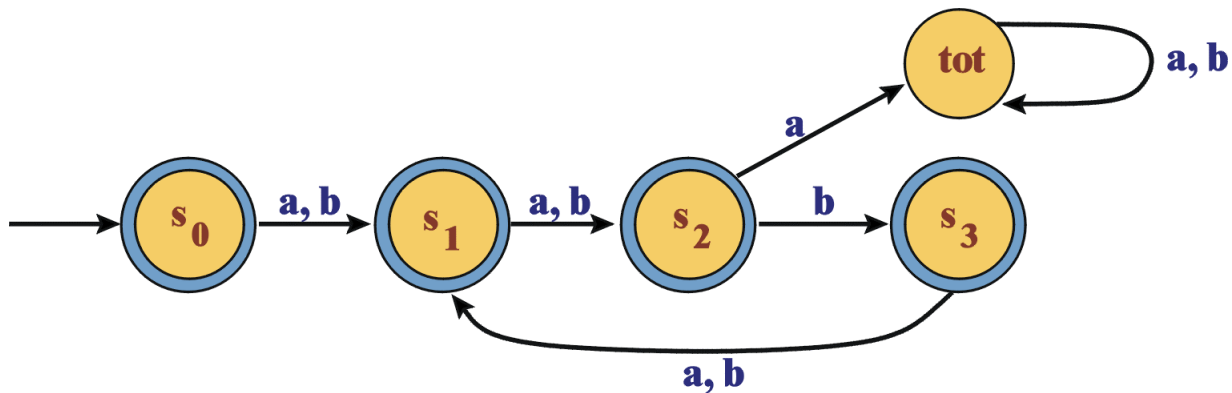
- Auch sämtliche Präfixe sollen erkannt werden:



Vollständige Automaten

- Bisher musste die Funktion δ eines DFA nicht total sein.
- Ein DFA $A = (\Sigma, S, \delta, s_0, F)$ heißt **vollständig**, wenn $\text{dom}(\delta) = S \times \Sigma$
- Jeder DFA $A = (\Sigma, S, \delta, s_0, F)$ kann durch Hinzunahme eines neuen Zustands *tot* vervollständigt werden:
Wenn $\delta(s, a)$ nicht definiert ist, ergänze $\delta(s, a) = \text{tot}$

Bsp:



Anwendungen für DFA: Suche in Texten

Verschiedene Szenarios:

Dynamische Texte

- Texteditoren
- Symbolmanipulatoren

Statische Texte

- Literaturdatenbanken
- Bibliothekssysteme
- Gen-Datenbanken
- WWW-Verzeichnisse

Datentyp **string**: array (file, list) of character

Operationen: (seien T, P von Typ **string**)

Länge: length (T)

***i*-tes Zeichen:** T [*i*]

Verkettung: cat (T,P) T.P

Problemdefinition: Mustersuche in Texten

Gegeben:

Text $t_1 t_2 \dots t_n \in \Sigma^n$

Muster $p_1 p_2 \dots p_m \in \Sigma^m$

Gesucht:

Ein oder alle Vorkommen des Musters im Text, d.h. Verschiebungen i mit $0 \leq i \leq n - m$ und

$$p_1 = t_{i+1}$$

$$p_2 = t_{i+2}$$

$$\vdots$$

$$p_m = t_{i+m}$$

Text: $\begin{array}{ccccccc} & & & i & & i+1 & & & i+m & & \\ \boxed{t_1} & \boxed{t_2} & \dots & & \boxed{t_{i+1}} & \dots & \boxed{t_{i+m}} & \dots \end{array}$

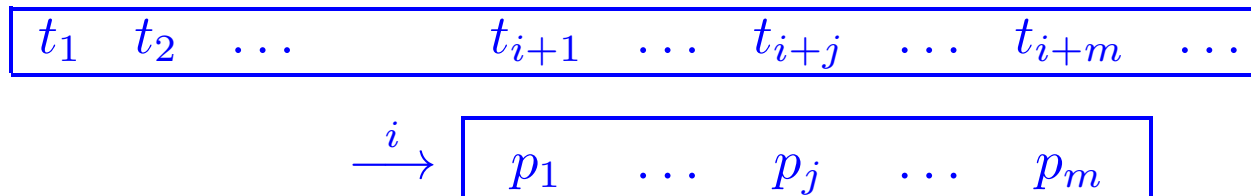
Muster: $\longrightarrow \boxed{p_1 \dots p_m}$

Aufwandsabschätzung (Zeit)?

Naives Verfahren

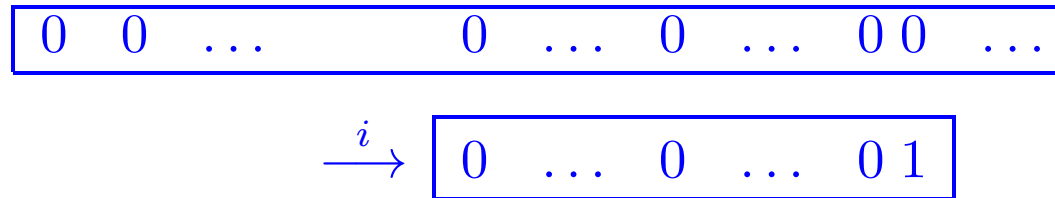
Für jede mögliche Verschiebung $0 \leq i \leq n - m$ prüfe maximal m Zeichenpaare.
Bei Mismatch beginne mit neuer Verschiebung.

```
textsearchbf := proc (T::string, P::string)
# Input:  Text T und Muster P
# Output: Liste L mit Verschiebungen i
#         an denen P in T vorkommt
  n := length (T); m := length (P);
  L := [];
  for i from 0 to n-m do
    j := 1;
    while j <= m and T[i+j] = P[j]
      do j := j+1 od;
    if j = m+1 then L := [L[],i] fi;
  od;
  RETURN (L)
end;
```



Abschätzung für naives Verfahren

Aufwandsabschätzung (Zeit):



Worst case: $\Omega(m n)$

In der Praxis oft: Mismatch tritt sehr früh auf

→ Laufzeit $\sim c n$

Verfahren nach Knuth-Morris-Pratt (KMP)

Seien t_i und p_{j+1} die zu vergleichenden Zeichen:

t_1	t_2	$\dots$	$\dots$	t_i	$\dots$	$\dots$						
		=	=	=	≠							
<table><tr><td>p_1</td><td>$\dots$</td><td>p_j</td><td>p_{j+1}</td><td>$\dots$</td><td>p_m</td></tr></table>							p_1	$\dots$	p_j	p_{j+1}	$\dots$	p_m
p_1	$\dots$	p_j	p_{j+1}	$\dots$	p_m							

Tritt bei einer Verschiebung erstmals ein Mismatch auf bei t_i und p_{j+1} , dann gilt:

- Die zuletzt verglichenen j Zeichen in T stimmen mit den ersten j Zeichen in P überein.
- $t_i \neq p_{j+1}$

Idee:

Bestimme $j' = \text{next}[j] < j$, so dass t_i anschliessend mit $p_{j'+1}$ verglichen werden kann.

Bestimme $j' < j$, so dass $P_{1\dots j'} = P_{j-j'+1\dots j}$.

Bestimme das längste Prefix von P , das echtes Suffix von $P_{1\dots j}$ ist.

Das next-Array beim KMP-Verfahren

Beispiel für die Bestimmung von $\text{next}[j]$:

$t_1 \ t_2 \dots \ 01011 \ 01011 \ 0 \ \dots$

01011 01011 1

01011 01011 1

$\text{next}[j]$ = Länge des längsten Prefixes von P , das echtes Suffix von $P_{1\dots j}$ ist.

$\Rightarrow$ für $P = 01011010111$ ist $\text{next} = [0, 0, 1, 2, 0, 1, 2, 3, 4, 5, 0]$:

1	2	3	4	5	6	7	8	9	10	11
0	1	0	1	1	0	1	0	1	1	1
		0								
		0	1							
					0					
					0	1				
					0	1	0			
					0	1	0	1		
					0	1	0	1	1	

Der KMP-Algorithmus

```
KMP := proc (T::string, P::string)
# Input:  Text T und Muster P
# Output: Liste L mit Verschiebungen i an denen P in T vorkommt
  n := length (T); m := length (P);
  L := [];
  next := KMPnext (P);
  j := 0;                # j: Zeiger vor Zeichen in P
  for i from 1 to n do    # i: Zeiger auf Zeichen in T
    while j>0 and T[i] <> P[j+1]
      do j := next [j] od;
    if T[i] = P[j+1] then j := j+1 fi;
    if j = m then L := [L[],i-m];
      j := next [j]
    fi;
  od;
  RETURN (L);
end;
```


Beispielanwendung für KMP-Algorithmus

Muster: abrakadabra, next = [0, 0, 0, 1, 0, 1, 0, 1, 2, 3, 4]

a b r a k a d a b r a b r a b a b r a k ...

| | | | | | | | | |

a b r a k a d a b r a

$next[11] = 4$

a b r a k a d a b r a b r a b a b r a k ...

- - - - ✗

a b r a k

$next[4] = 1$

a b r a k a d a b r a b r a b a b r a k ...

- | | | ✗

a b r a k

$next[4] = 1$

a b r a k a d a b r a b r a b a b r a k ...

- | ✗

a b r a k

$next[2] = 0$

a b r a k a d a b r a b r a b a b r a k ...

| | | | |

a b r a k

Korrektheit des Verfahrens nach Knuth-Morris-Pratt

Korrektheit:

$$\begin{array}{ccccccc}
 \boxed{t_1 & t_2 & \dots & & \dots & & t_i & \dots & \dots} \\
 & & & = & = & = & \neq & & \\
 & & & \boxed{p_1 & \dots & p_j & p_{j+1} & \dots & p_m}
 \end{array}$$

Situation am Beginn der for-Schleife: $P_{1\dots j} = T_{i-j\dots i-1}$ und $j \neq m$

falls $j = 0$: man steht auf erstem Zeichen von P

falls $j \neq 0$: P kann verschoben werden, solange $j > 0$ und $t_i \neq p_{j+1}$

Ist dann $T[i] = P[j + 1]$, können j und i (am Schleifenende) erhöht werden.

Wurde ganz P verglichen ($j = m$), ist eine Stelle gefunden, und es kann verschoben werden.

Laufzeit des KMP-Verfahrens

Laufzeit:

- Textzeiger i wird nie zurückgesetzt
- Textzeiger i und Musterzeiger j werden stets gemeinsam inkrementiert
- Es ist $\text{next}[j] < j$; j kann nur so oft herabgesetzt werden, wie es heraufgesetzt wurde.

Der KMP-Algorithmus kann in Zeit $O(n)$ ausgeführt werden, wenn das `next`-Array bekannt ist.

Zur Berechnung des next-Arrays

$\text{next}[i]$ = Länge des längsten Prefixes von P , das echtes Suffix von $P_{1\dots i}$ ist.

- $\text{next}[1] = 0$
- Sei $\text{next}[i-1] = j$:

p_1	p_2	$\dots$		$\dots$	p_{i-1}	p_i	$\dots$	$\dots$
			=	=	=	=	=?	
p_1	$\dots$	p_j	p_{j+1}	$\dots$	p_m			

Betrachte 2 Fälle:

- 1) $p_i = p_{j+1} \Rightarrow \text{next}[i] = j + 1$
- 2) $p_i \neq p_{j+1} \Rightarrow$ ersetze j durch $\text{next}[j]$, bis $p_i = p_{j+1}$ oder $j = 0$.
 Falls $p_i = p_{j+1}$ ist, kann $\text{next}[i] = j + 1$ gesetzt werden, sonst ist $\text{next}[i] = 0$

Berechnung des next-Arrays und Abschätzung

```
KMPnext := proc (P::string)
# Input:  Muster P
# Output: next-Array fuer P
  m := length (P);
  next := array (1..m);
  next [1] := 0;
  j := 0;
  for i from 2 to m do
    while j > 0 and P[i] <> P[j+1]
      do j := next [j] od;
    if P[i] = P[j+1] then j := j+1 fi;
    next [i] := j
  od;
  RETURN (next);
end;
```

Der KMP-Algorithmus kann in Zeit $O(n + m)$ ausgeführt werden.